Assignment 4 Sudoku Solver GenAI Analysis

1. GenAI Used
   The two Generative AIs that I used for this assignment are Perplexity AI accessed through its website (perplexity.ai) through the base model and Google Search AI accessed through the AI search on google.com
2. Code Analysis
   Correctness: Both programs implement object-oriented C++ classes to solve five Sudoku puzzles using recursive depth-first search with backtracking. Google Search AI uses a single SudokuBoard class managing grid arrays and recursion internally while Perplexity AI uses a modular approach separating concerns into SudokuPuzzle and SudokuApplication classes. Both handle file ingestion, check row, column, and block constraints, validate initial pre-filled clues, and print all unique valid solutions or "No solution found".

   Execution Time: Both programs have an exponential worst-case time complexity of O(9^m) for solving empty cells, but run efficiently in a fraction of a second due to pre-filled constraints and pruning during recursive backtracking. This is the expected execution time for brute-force Sudoku solvers.

   Space Complexity: Both use O(1) auxiliary space for storing grid data and bounded recursion stack depth corresponding to the 81 cells. Neither uses dynamic heap allocations during recursive search functions.

   Maintainability: Google Search AI encapsulates board operations within a single class using standard array storage and straightforward member functions. Perplexity AI does a better job of encapsulating the code through object-oriented programming by dividing responsibilities between puzzle management and batch application control. Both codes offer room for improvement on their maintainability.

3. GenAI Used
   I selected the Perplexity program as the basis for my assignment. It more efficiently uses separate classes for better object-oriented modularity and batch application control. It correctly prioritizes clean input validation and separates puzzle state logic from file processing. Its separate methods make the code easy to understand and modify. It also allows me to write the comments as opposed to code that already has comments.

4. Improvements
   Correctness: Ensured complete and accurate parsing of 81-cell puzzle files with underscore representations for blanks and digits 1 through 9 for givens. Verified that all unique solutions are correctly printed or "No solution found" is reported.

   Execution Time: O(1) auxiliary space and efficient constant-time constraint checks during recursive backtracking keep execution speeds fast across all test puzzle files.

   Space Complexity: Memory is kept efficient by using fixed-size grid representations and avoiding dynamic heap allocations during recursive search branches to save stack space.

Maintainability: Added prologue comments containing all required details, and added inline comments to each line to better the readability of the code. Inline comments are added to each line to help analyze each line and explain to the reader what the code is doing.

5. Raw Code

This is the code that immediately came out of my prompt. The same prompt was used for both GenAI which is just the standalone [Assignment 4 Instructions.pdf] upload followed by a prompt "lets get started" for both Programs.

Raw Google Search AI Code

```
/**
 * @file Assignment4.cpp
 * @brief Fully encapsulated Object-Oriented C++ Sudoku Solver using
Backtracking DFS.
 * @author [Your Full Name]
 * @date October 13, 2026
 *
 * Course: EECS 348 - Software Engineering I
 * Assignment: Assignment No. 4
 * Inputs: Nine-by-nine Sudoku text grids (puzzle1.txt to
puzzle5.txt) with '_' for empty cells.
 * Outputs: Prints the filename, original grid state, and all unique
valid solved boards.
 * Collaborators: [Placeholders for Collaborators]
 * Sources: Developed with architectural assistance from AI to
fulfill the GenAI evaluation requirement.
 */

#include <iostream>
#include <fstream>
#include <vector>
#include <string>
#include <cctype>

class SudokuBoard {
private:
    char grid[9][9];
    bool isOriginal[9][9];
    int solutionCount;
```

```
    // Checks if a candidate digit can be safely placed at
grid[row][col]
    bool isValid(int row, int col, char num) const {
        for (int i = 0; i < 9; ++i) {
            // Check row constraint
            if (grid[row][i] == num) return false;
            // Check column constraint
            if (grid[i][col] == num) return false;
        }

        // Check 3-by-3 block subgrid constraint
        int boxRowStart = (row / 3) * 3;
        int boxColStart = (col / 3) * 3;
        for (int i = 0; i < 3; ++i) {
            for (int j = 0; j < 3; ++j) {
                if (grid[boxRowStart + i][boxColStart + j] == num)
return false;
            }
        }
        return true;
    }

    // Recursive depth-first search solver function
    void solveCell(int row, int col) {
        // If we step past row 8, we have successfully filled out
the entire board
        if (row == 9) {
            solutionCount++;
            std::cout << "\nSolution " << solutionCount << ":\n";
            printGrid();
            return;
        }

        // Advance to the next row if we exceed the right-most
column
        if (col == 9) {
            solveCell(row + 1, 0);
            return;
        }
```

```
        // Skip cells that contained given constraints from the file
        if (grid[row][col] != '_') {
            solveCell(row, col + 1);
            return;
        }

        // Brute-force try digits '1' through '9'
        for (char num = '1'; num <= '9'; ++num) {
            if (isValid(row, col, num)) {
                grid[row][col] = num;       // Assign tentatively
                solveCell(row, col + 1);    // Recurse to next cell
                grid[row][col] = '_';       // Reset state
(backtrack)
            }
        }
    }

public:
    SudokuBoard() : solutionCount(0) {
        clearBoard();
    }

    void clearBoard() {
        solutionCount = 0;
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                grid[i][j] = '_';
                isOriginal[i][j] = false;
            }
        }
    }

    // Handles file ingestion, parsing logic, and integrity
verification
    bool loadFromFile(const std::string& filename) {
        clearBoard();
        std::ifstream inFile(filename);
        if (!inFile.is_open()) {
            std::cerr << "Error: Could not open file " << filename
<< "\n";
```

```
            return false;
        }

        char ch;
        int cellCount = 0;
        while (inFile >> ch) {
            if (ch == '_' || (ch >= '1' && ch <= '9')) {
                if (cellCount >= 81) {
                    std::cerr << "Malformed Input Error: File " <<
filename << " contains more than 81 board cells.\n";
                    return false;
                }
                int r = cellCount / 9;
                int c = cellCount % 9;
                grid[r][c] = ch;
                if (ch != '_') {
                    isOriginal[r][c] = true;
                }
                cellCount++;
            } else {
                std::cerr << "Malformed Input Error: Invalid
character '" << ch << "' encountered in " << filename << ".\n";
                return false;
            }
        }

        if (cellCount < 81) {
            std::cerr << "Malformed Input Error: File " << filename
<< " has incomplete data (found " << cellCount << "/81 cells).\n";
            return false;
        }

        // Validate that initial pre-filled cells do not violate
basic Sudoku rules
        for (int r = 0; r < 9; ++r) {
            for (int c = 0; c < 9; ++c) {
                if (isOriginal[r][c]) {
                    char temp = grid[r][c];
                    grid[r][c] = '_'; // Temporarily lift out cell
value to run standard check
```

```
                    if (!isValid(r, c, temp)) {
                        std::cerr << "Conflicting Given Digits
Error: Inherent duplicate values found in " << filename << ".\n";
                        return false;
                    }
                    grid[r][c] = temp; // Restore verified hint
                }
            }
        }
        return true;
    }

    void printGrid() const {
        for (int i = 0; i < 9; ++i) {
            for (int j = 0; j < 9; ++j) {
                std::cout << grid[i][j] << " ";
            }
            std::cout << "\n";
        }
    }

    // Encapsulated control sequence managing execution state
boundaries
    void processPuzzle(const std::string& filename) {
        std::cout << "========================================\n";
        std::cout << "Processing File: " << filename << "\n";

        if (!loadFromFile(filename)) {
            std::cout << "Skipping file due to execution errors.\n";
            std::cout <<
"========================================\n\n";
            return;
        }

        std::cout << "Original Puzzle Layout:\n";
        printGrid();

        solveCell(0, 0);

        if (solutionCount == 0) {
```

```
            std::cout << "\nNo solution found\n";
        }
        std::cout << "========================================\n\n";
    }
};

int main() {
    std::string puzzleFiles[] = {"puzzle1.txt", "puzzle2.txt",
"puzzle3.txt", "puzzle4.txt", "puzzle5.txt"};
    SudokuBoard solver;

    for (const std::string& file : puzzleFiles) {
        solver.processPuzzle(file);
    }

    return 0;
}
```

Raw Perplexity Code

```
/*
 * Program: EECS 348 Assignment 4 - Object-Oriented Sudoku Solver
 * Description: Read five Sudoku puzzles and print every solution
using
 *              recursive depth-first search with backtracking.
 * Inputs: puzzle1.txt, puzzle2.txt, puzzle3.txt, puzzle4.txt,
puzzle5.txt.
 *         Each file contains exactly 81 non-whitespace characters:
 *         digits 1-9 for given cells and underscores for empty
cells.
 * Outputs: Filename, original puzzle, and every solution; if there
are no
 *          solutions, print "No solution found". Report input
errors.
 * Author: [Enter your full name]
 * Collaborators: [List collaborators, or enter "None" if accurate]
 * Code source: Perplexity Computer, generated in response to the
user's
 *              EECS 348 Assignment 4 solver prompt.
 * Creation date: October 13, 2026
 *
```

```
 * Attribution: All implementation blocks below were generated by
 * Perplexity Computer. Record any later modifications and their
authors.
 * Review and rewrite explanatory comments in your own words as
required
 * by the assignment; do not claim generated code as independently
authored.
 */

#include <array>     // Fixed-size containers hold grids and
filenames.
#include <fstream>   // File streams read each puzzle from disk.
#include <iostream>  // Standard output displays puzzles and
diagnostics.
#include <string>    // Strings hold filenames and explanatory
errors.

// Source: Perplexity Computer. Encapsulates one puzzle and its
entire search.
class SudokuPuzzle {
private:
    enum { SIZE = 9, BOX_SIZE = 3, CELL_COUNT = SIZE * SIZE };
    using Grid = std::array<std::array<int, SIZE>, SIZE>;

    Grid original_{};       // Preserve the givens separately from
the search.
    Grid board_{};          // Store blanks as zero and filled cells
as 1-9.
    bool foundSolution_ = false; // Track existence without storing
solutions.

    // Source: Perplexity Computer. Print a supplied grid without
changing it.
    void printGrid(const Grid& grid, std::ostream& out) const {
        for (int row = 0; row < SIZE; ++row) { // Visit each row in
order.
            for (int col = 0; col < SIZE; ++col) { // Visit each
cell in the row.
                if (col != 0) {
```

```
                    out << ' '; // Separate entries with a single
space.
                }
                if (grid[row][col] == 0) {
                    out << '_'; // Retain the input convention for
blanks.
                } else {
                    out << grid[row][col]; // Print a filled cell's
digit.
                }
            }
            out << '\n'; // Finish the current row.
        }
    }

    // Source: Perplexity Computer. Check the three Sudoku
constraints.
    // Exclude the target cell so this works for givens as well as
blanks.
    bool canPlace(int row, int col, int value) const {
        for (int index = 0; index < SIZE; ++index) {
            if (index != col && board_[row][index] == value) {
                return false; // Another cell in this row already
has value.
            }
            if (index != row && board_[index][col] == value) {
                return false; // Another cell in this column has
value.
            }
        }

        const int firstRow = (row / BOX_SIZE) * BOX_SIZE;
        const int firstCol = (col / BOX_SIZE) * BOX_SIZE;
        for (int r = firstRow; r < firstRow + BOX_SIZE; ++r) {
            for (int c = firstCol; c < firstCol + BOX_SIZE; ++c) {
                if ((r != row || c != col) && board_[r][c] == value)
{
                    return false; // A different cell in this block
has value.
                }
```

```
            }
        }
        return true; // No row, column, or block conflict was found.
    }

    // Source: Perplexity Computer. Enumerate all completions
recursively.
    // Minimum remaining values (MRV) chooses a cell; it never drops
a legal
    // candidate and therefore does not change the set of solutions.
    void search(std::ostream& out) {
        int chosenRow = -1; // No blank has been selected yet.
        int chosenCol = -1;
        int fewestCandidates = SIZE + 1; // Any real count will be
smaller.
        std::array<int, SIZE> chosenCandidates{}; // Values for the
best cell.

        for (int row = 0; row < SIZE; ++row) {
            for (int col = 0; col < SIZE; ++col) {
                if (board_[row][col] != 0) {
                    continue; // Neither givens nor current guesses
are blanks.
                }
                std::array<int, SIZE> candidates{};
                int count = 0; // Count all legal values for this
empty cell.
                for (int value = 1; value <= SIZE; ++value) {
                    if (canPlace(row, col, value)) {
                        candidates[count++] = value; // Keep each
legal digit.
                    }
                }
                if (count == 0) {
                    return; // This partial board cannot be
completed.
                }
                if (count < fewestCandidates) {
                    chosenRow = row; // Prefer the most constrained
blank.
```

```
                    chosenCol = col;
                    fewestCandidates = count;
                    chosenCandidates = candidates;
                }
            }
        }

        if (chosenRow == -1) {
            foundSolution_ = true; // No blank remains, so the board
is solved.
            out << "Solution:\n";
            printGrid(board_, out); // Stream this result instead of
saving it.
            out << '\n';
            return; // Return to the caller to explore other
possible branches.
        }

        for (int index = 0; index < fewestCandidates; ++index) {
            board_[chosenRow][chosenCol] = chosenCandidates[index];
            search(out); // Explore every completion with this
candidate.
            board_[chosenRow][chosenCol] = 0; // Undo the guess
before continuing.
        }
    }

public:
    // Source: Perplexity Computer. Parse exactly 81 non-whitespace
cells.
    // A temporary grid prevents a malformed file from partially
loading state.
    bool load(const std::string& filename, std::string& error) {
        error.clear(); // Do not retain an error from an earlier
operation.
        original_ = Grid{}; // Reset all object state before loading
a new file.
        board_ = Grid{};
        foundSolution_ = false;
        std::ifstream input(filename);
```

```
        if (!input) {
            error = "Cannot open puzzle file.";
            return false; // The application will still process
later files.
        }

        Grid parsed{};
        int count = 0;
        char token = '\0';
        while (input >> token) { // Formatted extraction skips
whitespace.
            if (count >= CELL_COUNT) {
                error = "Expected exactly 81 cells; found extra
data.";
                return false;
            }
            if (token != '_' && (token < '1' || token > '9')) {
                error = "Invalid character at cell " +
std::to_string(count + 1)
                        + "; expected a digit 1-9 or underscore.";
                return false;
            }
            parsed[count / SIZE][count % SIZE] =
                (token == '_') ? 0 : token - '0'; // Map tokens to
numeric cells.
            ++count; // Advance to the next row-major cell.
        }
        if (input.bad() || (input.fail() && !input.eof())) {
            error = "An I/O error occurred while reading the file.";
            return false;
        }
        if (count != CELL_COUNT) {
            error = "Expected exactly 81 cells; found " +
std::to_string(count)
                    + ".";
            return false;
        }
        original_ = parsed; // Keep an untouched copy for original
output.
        board_ = parsed; // Initialize the working search board.
```

```
        return true;
    }

    // Source: Perplexity Computer. Reject conflicting givens before
searching.
    bool validate(std::string& error) const {
        error.clear();
        for (int row = 0; row < SIZE; ++row) {
            for (int col = 0; col < SIZE; ++col) {
                const int value = board_[row][col];
                if (value != 0 && !canPlace(row, col, value)) {
                    error = "Conflicting given at row " +
std::to_string(row + 1)
                            + ", column " + std::to_string(col + 1) +
".";
                    return false; // Givens cannot be changed to
remove a conflict.
                }
            }
        }
        return true; // Every nonzero given obeys the constraints.
    }

    // Source: Perplexity Computer. Expose printing without exposing
the grid.
    void printOriginal(std::ostream& out) const {
        printGrid(original_, out);
    }

    // Source: Perplexity Computer. Reset, validate, and enumerate
all solutions.
    // This method also protects callers that forgot to validate
first.
    void solveAll(std::ostream& out) {
        board_ = original_; // Repeated calls start with the same
original board.
        foundSolution_ = false;
        std::string error;
        if (!validate(error)) {
            out << "Error: " << error << '\n';
```

```
            out << "No solution found\n";
            return;
        }
        search(out);
        if (!foundSolution_) {
            out << "No solution found\n"; // Use the exact required
message.
        }
    }
};

// Source: Perplexity Computer. Coordinate the five required puzzle
files.
class SudokuApplication {
public:
    int run() const {
        const std::array<std::string, 5> filenames = {{
            "puzzle1.txt", "puzzle2.txt", "puzzle3.txt",
            "puzzle4.txt", "puzzle5.txt"
        }};
        bool inputError = false; // Remember errors without stopping
the loop.
        for (const std::string& filename : filenames) {
            std::cout << "Puzzle file: " << filename << '\n';
            SudokuPuzzle puzzle; // Isolate state for this file from
other files.
            std::string error;
            if (!puzzle.load(filename, error)) {
                std::cout << "Error: " << error << "\n\n";
                inputError = true;
                continue; // A bad or missing file does not stop
later puzzles.
            }
            std::cout << "Original puzzle:\n";
            puzzle.printOriginal(std::cout);
            std::cout << '\n';
            if (!puzzle.validate(error)) {
                std::cout << "Error: " << error << '\n';
                std::cout << "No solution found\n\n";
                inputError = true;
```

```
                continue; // Avoid searching a board with
conflicting givens.
            }
            puzzle.solveAll(std::cout); // Print all solutions or
the failure text.
            std::cout << '\n';
        }
        return inputError ? 1 : 0; // Unsolvable valid puzzles are
not input errors.
    }
};

// Source: Perplexity Computer. The required entry point delegates
to an object.
int main() {
    SudokuApplication application;
    return application.run();
}
```